

Mastering Dynamic Programming

An Introduction to Optimization, Complexity Reduction, and Solving Complex Algorithmic Challenges.

ALGORITHMS & DATA STRUCTURES SERIES

| What is Dynamic Programming?



Core Concept

Dynamic Programming (DP) is an algorithmic technique for solving problems by breaking them down into simpler **subproblems** and storing results to avoid redundant work.



The "Magic"

It transforms **exponential** time complexity into **polynomial** time by trading space for speed. It is the "smart" way to do recursion.

| The Origins



Richard Bellman (1920–1984)

The Inventor

Richard Bellman developed Dynamic Programming in the 1950s while working at RAND Corporation. He coined the term "Dynamic Programming" to hide the mathematical nature of his work from political scrutiny.

Today, his **Bellman Equation** remains the cornerstone of modern optimization and reinforcement learning.

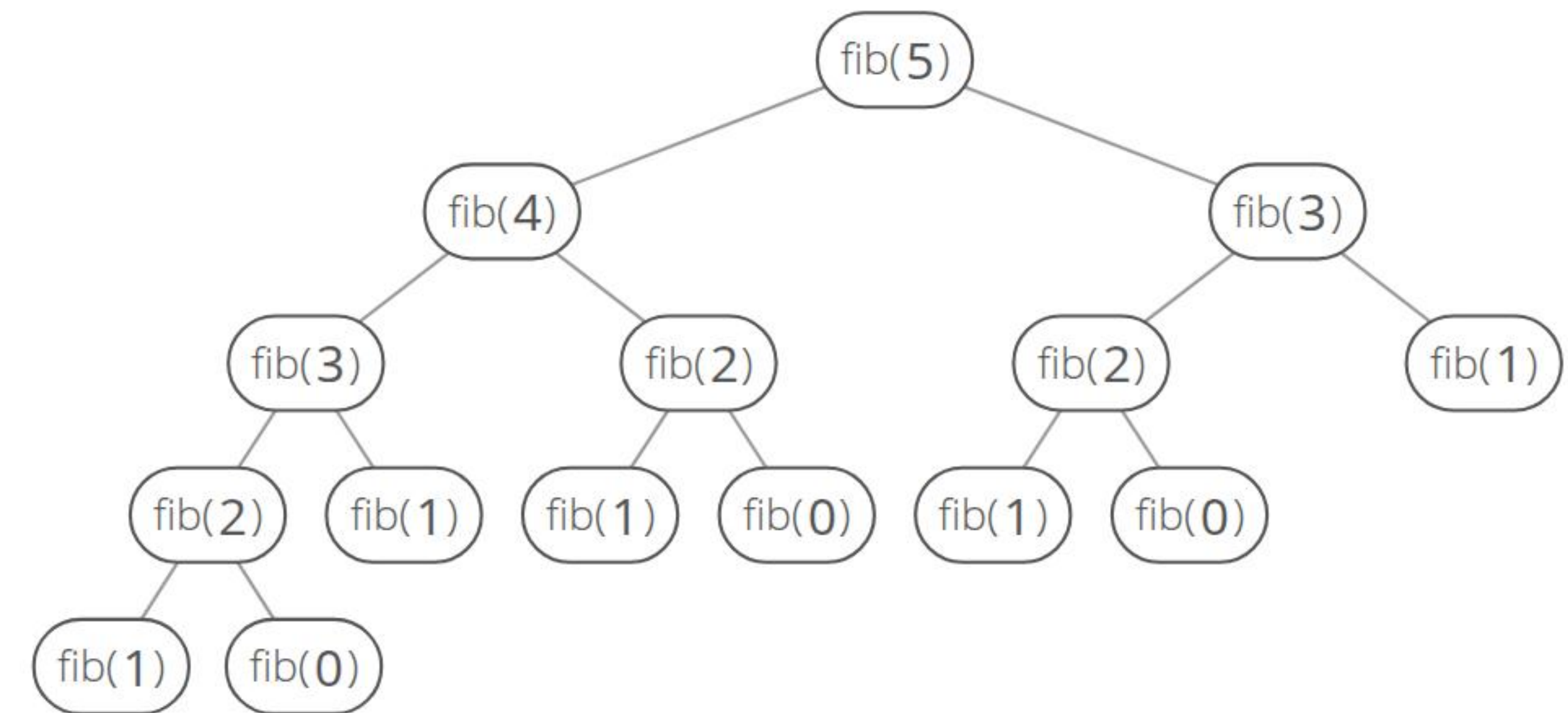
Key Property: Overlapping Subproblems

A problem has **Overlapping Subproblems** if it can be broken down into smaller subproblems which are reused multiple times.

🔗 Standard recursion computes the same subproblem repeatedly.

🗄️ DP solves each subproblem **only once**.

🏠 Results are stored in a table (Cache).



| Key Property: Optimal Substructure

-  **Definition:** The optimal solution to the main problem can be constructed from the optimal solutions of its subproblems.
-  **Shortest Path Example:** If the shortest path from A to C passes through B, then the segment from A to B must also be the shortest path between A and B.
-  **Important Note:** Not all problems have this. For example, finding the "Longest Simple Path" does not typically exhibit optimal substructure.

Top-Down Approach: Memoization



How it Works

Starts with the main problem and recursively breaks it down. Before solving a subproblem, it checks if it has been solved before.



Process

1. Check Cache
2. Solve & Save
3. Return Result



Best For

When you don't need to solve all subproblems to find the final answer. It is more intuitive for recursive thinkers.

Bottom-Up Approach: Tabulation



How it Works

Solves all subproblems starting from the smallest (base cases) and builds up to the target problem using an iterative table.



Process

1. Initialize Table
2. Set Base Cases
3. Fill Iteratively



Best For

Predictable memory usage and avoiding stack overflow from deep recursion. Usually faster in practice due to lack of recursion overhead.

The "Hello World" of DP: Fibonacci

The Fibonacci recurrence relation is the perfect showcase for DP's power.

$$F(n) = F(n - 1) + F(n - 2)$$

Approach	Time Complexity	Space Complexity
Recursion	$O(2^n)$	$O(n)$
DP (Tab)	$O(n)$	$O(n)$ or $O(1)$

Why is it better?

For $n=50$:

- **Recursion:** ~1.12 Quadrillion calls.
- **DP:** Exactly 50 operations.

From millions of years of computing to less than a millisecond.

| The 4-Step DP Recipe

- **1 Define the State:** What does your table entry represent? (e.g., $dp[i]$ = max profit at index i).
- **2 Recurrence Relation:** How do you calculate the current state using previous states?
- **3 Identify Base Cases:** What are the simplest subproblems you already know the answer to?
- **4 Decide Direction:** Will you use Top-Down (Recursion + Memo) or Bottom-Up (Loops)?

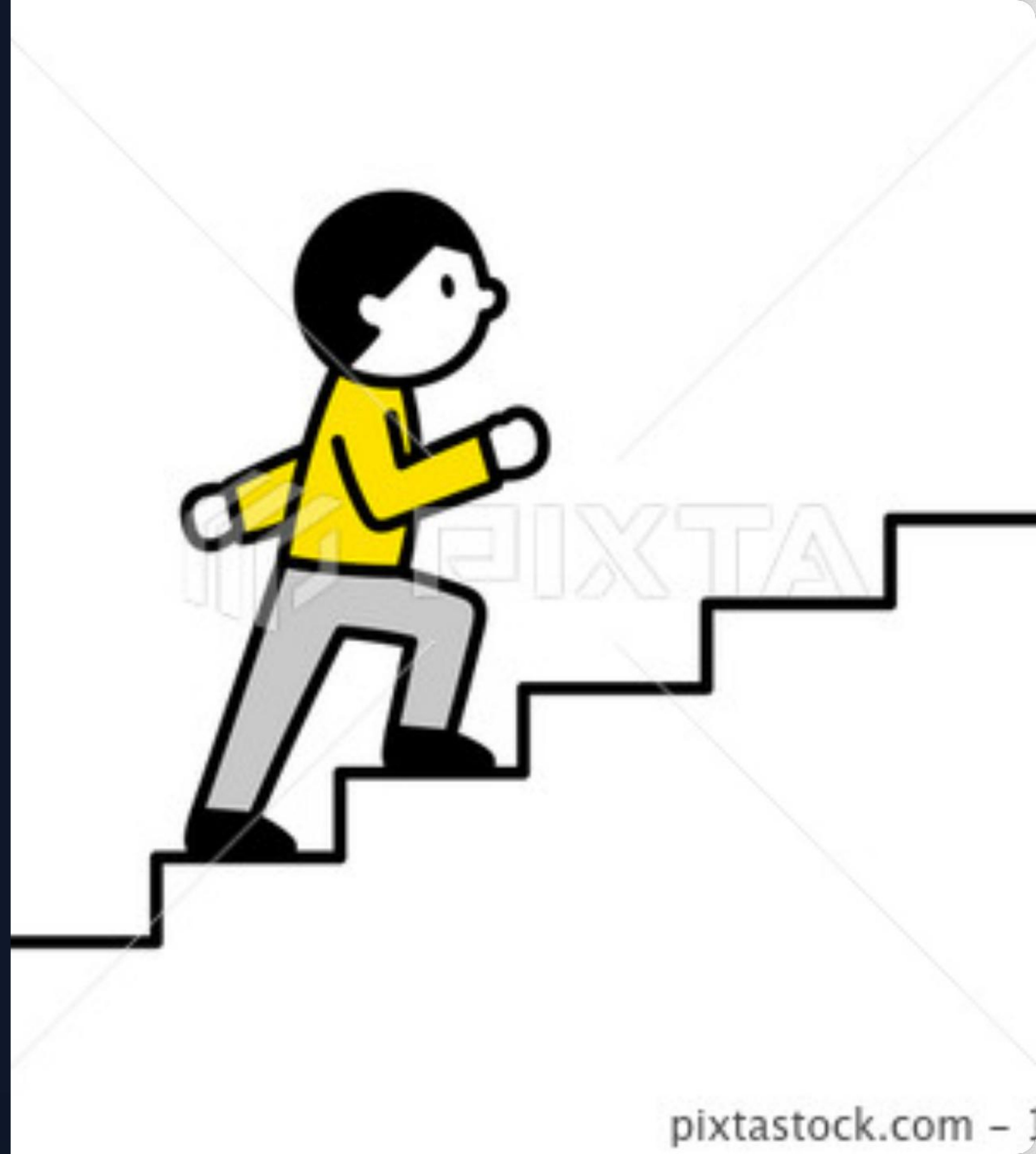
Problem 1: Climbing Stairs

Problem: You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps.

In how many distinct ways can you climb to the top?

Hint

To reach step i , you must have come from step $i-1$ or $i-2$. Does this look like a sequence we've seen?



Problem 2: Coin Change



The Challenge

Given an array of coin denominations and a total amount, find the **minimum** number of coins needed to make up that amount.

Example: Amount = 11, Coins = [1, 2, 5].

Answer: 3 (5 + 5 + 1)

State: $dp[i]$ = min coins for amount i .

Problem 3: House Robber

The Challenge

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed.

Constraint: Adjacent houses have security systems connected; you cannot rob two adjacent houses on the same night.

What is the maximum amount of money you can rob?



Q&A

Thank you for exploring Dynamic Programming basics!

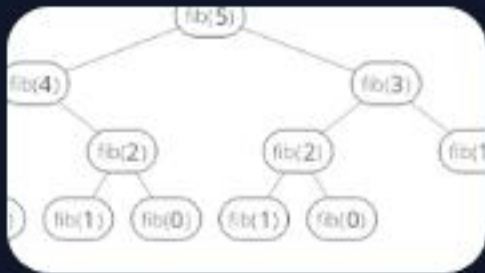
Ready to optimize your next solution?

Image Sources



https://mathshistory.st-andrews.ac.uk/Biographies/Bellman/bellman_4.jpg

Source: mathshistory.st-andrews.ac.uk



https://www.interviewcake.com/images/svg/fibonacci_binary_tree_recursive.svg?bust=211

Source: www.interviewcake.com



<https://en.pimg.jp/115/022/226/1/115022226.jpg>

Source: www.pixtastock.com



https://static.vecteezy.com/system/resources/previews/071/959/242/non_2x/metal-coins-of-ukraine-of-different-denominations-are-scattered-money-texture-business-background-photo.JPG

Source: www.vecteezy.com



<https://media.istockphoto.com/id/1419292229/vector/cozy-urban-autumn-landscape-cute-town-with-small-houses-architecture-of-city-street-exterior.jpg?s=612x612&w=0&k=20&c=sPF4DbD5JXpsDFRZeIQKEyFdbtyqe8ml6VfK3IGxIVo=>

Source: www.istockphoto.com